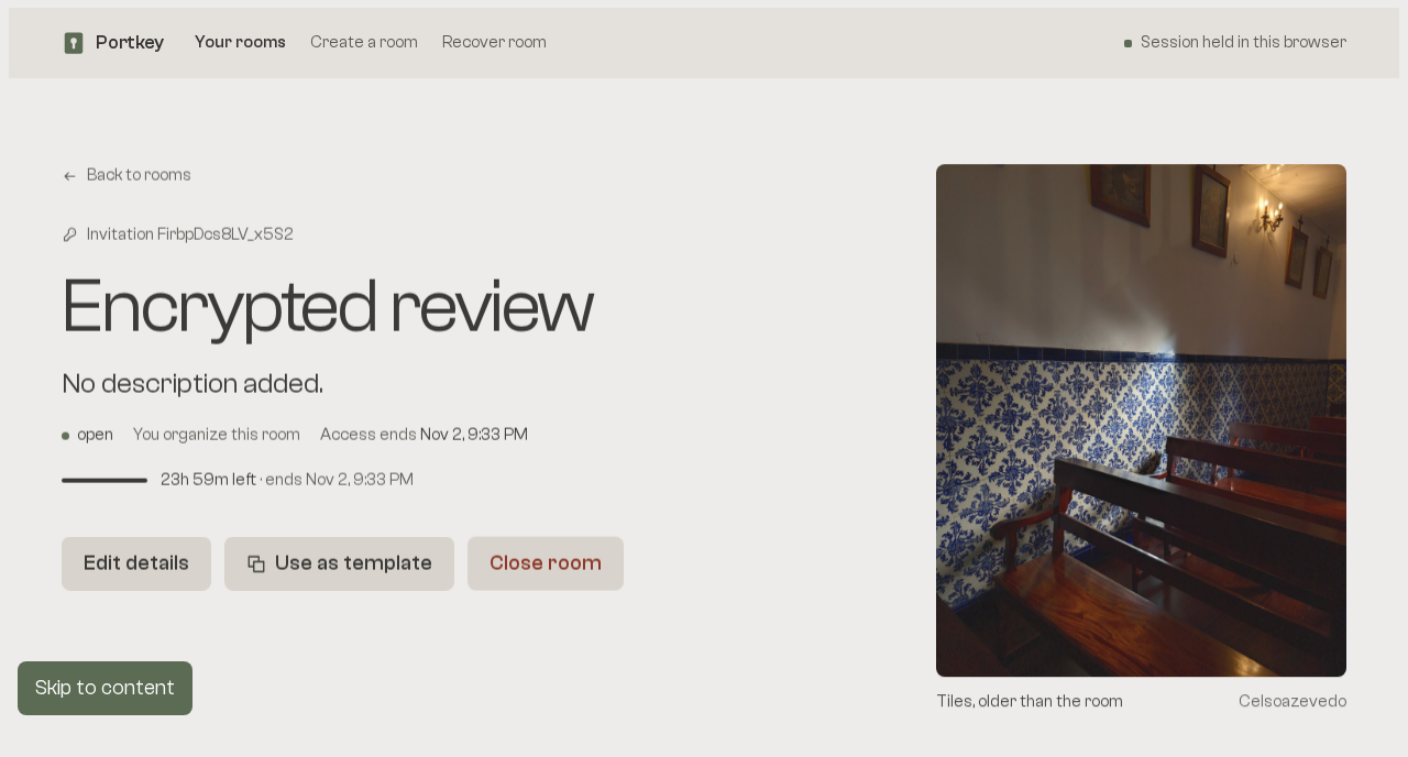
Portkey

Zero-knowledge credential admission and encrypted room file sharing

Full Stack Development project - implementation and feature mapping report

Updated 1 November 2026. React and TypeScript in the browser. Express with PostgreSQL or PGlite on the server.



Scope: organizers distribute files to approved participants. Participant uploads are deliberately excluded. The repository includes operational tooling, automated tests and reproducible evidence.

Readiness: the software paths below are implemented and locally verified. Institutional identity integration, independent ceremony participation, real-device acceptance and an external security review remain operator responsibilities.

Architecture and trust boundaries

Browser: session UI, credential proof generation, HPKE key handling, AES-GCM content encryption, resumable upload journal and authenticated downloads.

API: session and CSRF checks, room lifecycle, challenge consumption, proof verification, admission decisions, key/envelope policies, quotas, rate budgets and cleanup.

Database: sessions, rooms, memberships, room activity, public recipient keys, ciphertext chunks/envelopes, credential challenges/revocations, one-time recovery hashes and shared rate windows.

Data	Browser	API/database
File name and contents	Plaintext during selection/decryption	Encrypted descriptor and chunks
Private decryption key	IndexedDB / passphrase-protected backup	Never uploaded
Credential secret and signature	Private proof inputs	Never uploaded by normal join flow
Eligibility proof	Generated locally	Proof and nine public signals
Location, labels, sizes and timings	Visible locally	Visible to service / organizer as authorized

A compromised server can serve altered JavaScript that steals browser secrets. This design protects stored content under the intended application, not against a malicious application origin. Browser location remains spoofable.

Database-clock expiry and room locks make admission, publication and cleanup policies consistent. API responses use no-store caching; writes require Origin and CSRF checks. Issuer revocations use pinned-issuer signature authorization.

Credential proof protocol v2

The issuer signs Poseidon(secret, department, class year, validUntil). The circuit proves signature possession and derives both a per-room nullifier and a binding to a fresh random session challenge.

Public signal	Purpose
nullifier	One credential seat per room
binding	Poseidon(secret, challenge) constrains the challenge in the circuit
issuerAx, issuerAy	Pinned institution public key
department, year	Eligibility requirement
scope	Room UUID interpreted as a field value
challenge	Random ten-minute value owned by the requesting session/room
validUntil	Signed credential expiry checked using server/database time

The server checks the pairing, issuer, attributes and scope, then checks the unconsumed session challenge, expiry and revocation under the room lock. Successful admission consumes the challenge atomically. A copied proof cannot front-run from a different browser session.

Issuer-signed revocations publish separately scoped nullifiers for affected rooms and block both future joins and existing access. The supplied institution-side worker/timer applies that policy to affected current and future rooms on a schedule; there is no public cross-room credential identifier. The delivery interval remains a revocation-latency boundary.

Migration 010 returns approved v1 credential memberships to pending. Operators must issue v2 credentials and deploy the matching wasm, zkey, verification key and checksum manifest together.

File encryption, resume and key management

Each publication uses a fresh secret and salt. Separate HKDF-SHA256 labels derive AES-256-GCM content and metadata keys. One-MiB chunks use indexed nonces and context-bound authenticated data. HPKE P-256/HKDF-SHA256/AES-256-GCM seals the file key to each recipient.

The server accepts immutable chunk retries and verifies a SHA-256 descriptor/chunk manifest on completion. Downloads authenticate every chunk and manifest before the final save. Receipts record server acceptance and are not independent signed attestations.

Capability	Implementation
250 MiB files	250 chunks, 1,000 files and 5 GiB encrypted chunk storage per room
Durable resume	Immutable source snapshot in OPFS or bounded 50 MiB memory-storage fallback; plan/chunks in IndexedDB
Nonce safety	Persist reservation before encryption and ciphertext before upload; abandon interrupted reservations
Late arrivals	Organizer grants new envelopes over existing ciphertext
Key rotation	Atomically rewrap every current envelope; reject stale keys or incomplete replacement sets
Ownership recovery	Separate one-time random recovery code transfers authority; encrypted key backup restores decryption
Deletion	Owner can delete completed files; expiry cleanup cascades ciphertext/envelopes/chunks

OPFS-unavailable browsers use bounded fallback behavior. Browser eviction can remove pending state. Rotation and revocation do not erase keys, ciphertext or plaintext already copied by a recipient.

Feature mapping and implementation files

Feature	Main implementation	Verification
Room lifecycle / approval	server/rooms.ts; client/RoomPage.tsx	API + mobile browser flows
Proof challenge and expiry	circuits/credential.circom; server/zk.ts	Real Groth16 proofs and replay/expiry cases
Institution revocation	server/issuerRoutes.ts; scripts/issuer.ts	Pinned issuer signature and revocation cases
Encrypted file distribution	server/roomFiles.ts; client/roomCrypto.ts	HPKE vector + browser decrypt/integrity
Resume / cancellation	client/uploadStore.ts; client/RoomFiles.tsx	Reload/resume and reserved-nonce faults
Recovery / rotation / grants	server/recovery.ts; server/keys.ts	API + ownership/decryption browser flow
Geofence	server/participantAccess.ts	Direct-route bypass regression
Shared rate budgets	server/rateStore.ts	Two-instance concurrency / PostgreSQL load probe
Operations / deployment	server/operations.ts; compose.yaml; deploy/	Metrics auth, restore probe, HTTPS smoke
Accessibility / compatibility	e2e/accessibility.spec.ts; compatibility.spec.ts	Axe serious/critical checks; browser matrix

Schema history: 001-006 establish rooms, history, historical submissions and geofencing. 007 adds room file distribution; 008 removes old submission tables; 009 adds credential gates; 010 adds challenge/expiry/revocation/recovery/shared budgets and privacy cleanup indexes.

No applied historical migration was rewritten. Migration 010 is a new migration. The old submission data removal in migration 008 requires backup and an intentional upgrade decision.

Measured verification results

Check	Evidence
API, embedded PostgreSQL	62 passing tests on PGlite
API, PostgreSQL 17	62 passing tests against a real PostgreSQL container
Browser workflows	11/11 Chromium workflow tests passed in the final combined suite. The final upload-plan fingerprint check also passed its targeted API/browser regressions.
Compatibility	9 cases passed across Chromium, Firefox and WebKit engines, including real credential proofs. Actual phones and shipping Safari still need acceptance testing.
Production	Three real proof/key-lifecycle flows through built assets, HTTPS, secure cookies and CSP
Accessibility	Workspace, room and recovery form: no serious/critical Axe WCAG A/AA violations
Resilience	Offline/reconnect, no OPFS, quota failure and abandoned nonce reservation
Backup / restore	Migrated app tables and exact ciphertext bytes restored into a new database

250 MiB end-to-end transfer

262,144,000 plaintext bytes, 250 chunks, 262,148,000 encrypted chunk bytes. The test verified every saved plaintext byte. Publication took 98.2 seconds and download/decryption took 34.2 seconds on this machine using PGlite and headless Chromium.

A separate two-instance PostgreSQL rate-limit probe sent 200 requests at concurrency 20. Exactly 30 new sessions succeeded and 170 received 429. Observed p95 latency was 103.2 ms. This is a budget-coordination probe, not a production capacity guarantee.

Evidence: benchmark-results/250-mib.json, postgres-load.json, restore.json and docs/evidence.json. Timings depend on this machine, storage and concurrent test activity. No real-phone or production traffic benchmark is claimed.

Deployment and remaining external actions

The supplied Dockerfile runs as an unprivileged Node user. compose.yaml supplies PostgreSQL readiness, persistence and a loopback API port. Caddy configuration terminates HTTPS. Proxy hop trust is explicit. PostgreSQL coordinates rate budgets across API instances.

Health and token-protected metrics report storage and cleanup backlog. Cleanup purges expired file rows, reclaims incomplete uploads after 24 hours and clears proximity fields after 24 hours. Backups and downloaded copies follow their own retention policies.

Required institutional actions

1. Supply and review an independent external ceremony for this v2 circuit. Import verifies the ptau/zkey relationship and hashes, but cannot prove entropy destruction or replace independent participants.

2. Supply an institution-held issuer key, authenticated roster/credential delivery channel, issuer rotation policy and a worker that applies scoped revocations across affected rooms. No real institution credentials or student records were invented.

3. Select a production domain/host, configure secrets and HTTPS, run the supplied CI on the actual repository, set monitoring thresholds and exercise the institution's backup/restore policy.

4. Run acceptance tests on actual phones and supported Safari versions, and commission independent cryptographic/application security review before relying on real student eligibility.

The software and local evidence are available now. External deployment, institutional onboarding and an independent ceremony cannot be truthfully marked complete in this local session. Participant uploads remain excluded by request.

Sources and reproduction

README.md, DEPLOYMENT.md, AUDIT.md, numbered SQL migrations, e2e/ and tests/, and the evidence JSON files. Ceremony procedure follows the primary snarkjs documentation: https://github.com/iden3/snarkjs#readme.